

Trabajo Final de Asignatura

Máster Universitario en Inteligencia de Datos y Ciberseguridad

Visualización

Análisis de la desigualdad económica
en la provincia de Santa Cruz de Tenerife

Francisco Marqués Armas

La Laguna, 24 de mayo de 2026

Índice general

1. Introducción y descripción del conjunto de datos	1
1.1. Motivación	1
1.2. Fuentes de datos	1
1.3. Preguntas de investigación	1
2. Gramática de gráficos	3
2.1. Lollipop: ranking de municipios por renta media	3
2.2. Histograma: distribución de la renta por sección censal	4
2.3. Waterfall: fuentes de ingresos	5
2.4. Waterfall: distribución por sector de actividad	6
2.5. Diverging bar: brecha de género en ocupaciones	7
2.6. Slope chart: evolución de la renta 2021–2023	8
2.7. Box plots: desigualdad intramunicipal por isla	8
2.8. Scatter: estudios elementales vs. renta	10
2.9. Grouped bar: distribución de actividad por quintil de renta	11
2.10. Grouped bar: distribución de ocupación por quintil de renta	12
2.11. Heatmap: evolución de la renta municipal 2021–2023	13
2.12. Mapa interactivo: secciones censales	14
2.12.1. Técnica y datos	14
2.12.2. Codificación visual	14
2.12.3. Diseño geoespacial vs. gramática de gráficos	14
3. Principios de diseño y gestalt	15
3.1. Recomendaciones de diseño aplicadas	17
4. Arquitectura DataOps con Dagster	18
4.1. Motivación	18
4.2. Grafo de assets	18
4.3. Generación de código con IA y caché de prompts	18
4.4. Beneficios de la arquitectura	19
5. Checks de calidad de assets	21
5.1. Clasificación de los checks	21
5.1.1. Checks de carga (integridad básica)	21
5.1.2. Checks de limpieza	21
5.1.3. Checks analíticos	21
5.1.4. Checks de integridad de archivos generados	22
5.2. Metadatos de los checks	22
6. Dashboard interactivo	24
6.1. Motivación	24
6.2. Estructura y narrativa	24
6.3. Integración del mapa interactivo	24
6.4. Navegación y estilo	25
6.5. Arranque	25

1. Introducción y descripción del conjunto de datos

1.1. Motivación

La provincia de Santa Cruz de Tenerife, que comprende las islas de Tenerife, La Palma, La Gomera y El Hierro, presenta una distribución de la renta notablemente desigual: municipios turísticos y de servicios conviven con zonas de actividad agrícola o poca actividad económica cuya renta per cápita es significativamente inferior. Este proyecto aplica las técnicas de visualización de datos aprendidas en la asignatura para explorar, comunicar y cuantificar esa desigualdad a través de datos oficiales del Instituto Nacional de Estadística (INE).

1.2. Fuentes de datos

El proyecto emplea cuatro conjuntos de datos, todos de acceso público mediante la API JSON-STAT del INE:

Dataset	Variable clave	Cobertura	Granularidad
Renta media neta	OBS_VALUE (euros/hogar)	2021–2023	Sección censal
Fuentes de ingresos	Porcentaje por fuente	2023	Municipio
Actividad económica	Número de trabajadores	2023	Municipio por sector
Categoría de ocupación	Número de trabajadores	2023	Municipio, ocupación y sexo

Tabla 1.1: Conjuntos de datos utilizados. Los cuatro proceden del INE y cubren la provincia de Santa Cruz de Tenerife (código INE: 38).

- Renta media: Atlas de distribución de renta de los hogares (ADRH), indicador de renta neta media por hogar a nivel de sección censal. Permite comparar municipios y analizar la desigualdad intramunicipal.
- Fuentes de ingresos: porcentaje de la renta total que procede de sueldos y salarios, pensiones, prestaciones por desempleo, otras prestaciones sociales y otros ingresos.
- Actividad económica: distribución de los asalariados por sector CNAE: Agricultura/pesca, Construcción, Industria y Servicios.
- Categoría de ocupación: distribución de asalariados por categoría de ocupación (CNO) y sexo, lo que permite estudiar la brecha de género.

1.3. Preguntas de investigación

Las visualizaciones se organizan en torno a tres preguntas:

1. ¿Cómo se distribuye la renta entre los municipios? ¿Hay diferencias sistemáticas entre islas? ¿Ha evolucionado la brecha en los últimos tres años?

2. ¿Qué caracteriza económicamente a los municipios ricos y a los pobres? ¿En qué sectores trabajan, qué ingresos reciben y qué ocupaciones predominan en los extremos de la distribución?
3. ¿Qué desigualdad existe dentro de cada municipio? ¿Qué secciones censales concentran la riqueza y cuáles la pobreza?

2. Gramática de gráficos

Este capítulo analiza cada visualización generada siguiendo la gramática de gráficos de Wilkinson [1] tal como la implementa Plotnine [2]: datos, mapeos estéticos (aes), geometrías (geom_*), escalas, coordenadas, facetas y tema.

2.1. Lollipop: ranking de municipios por renta media



Figura 2.1: Lollipop de ranking de municipios por renta media neta (2023).

- Datos: tabla con un registro por municipio; columnas municipio (cadena) y renta_media (numérico, euros).
- Mapeos:
 - x = municipio (variable categórica, ordenada por renta de menor a mayor)
 - y = renta_media (variable cuantitativa continua)
 - color = isla (variable categórica nominal; distingue visualmente a qué isla pertenece cada municipio)
- Geometrías: geom_segment (el palo del lollipop, desde y=0 hasta el valor) + geom_point (la cabeza del lollipop).
- Escalas:
 - Eje Y: escala continua en euros; etiquetas con separador de miles.
 - Color: escala discreta cualitativa (cuatro colores, uno por isla).
- Coordenadas: coord_flip() para orientar las barras horizontalmente y hacer legibles los nombres de municipio.

- Facetas: ninguna; todos los municipios en un único panel.
- Tema: `theme_minimal()` con texto del eje Y reducido (`size=7`) para acomodar 54 etiquetas.

El lollipop es preferible al gráfico de barras cuando hay muchas categorías: la línea delgada reduce la tinta y la comparación de valores se realiza rápidamente gracias al punto claramente posicionado.

2.2. Histograma: distribución de la renta por sección censal

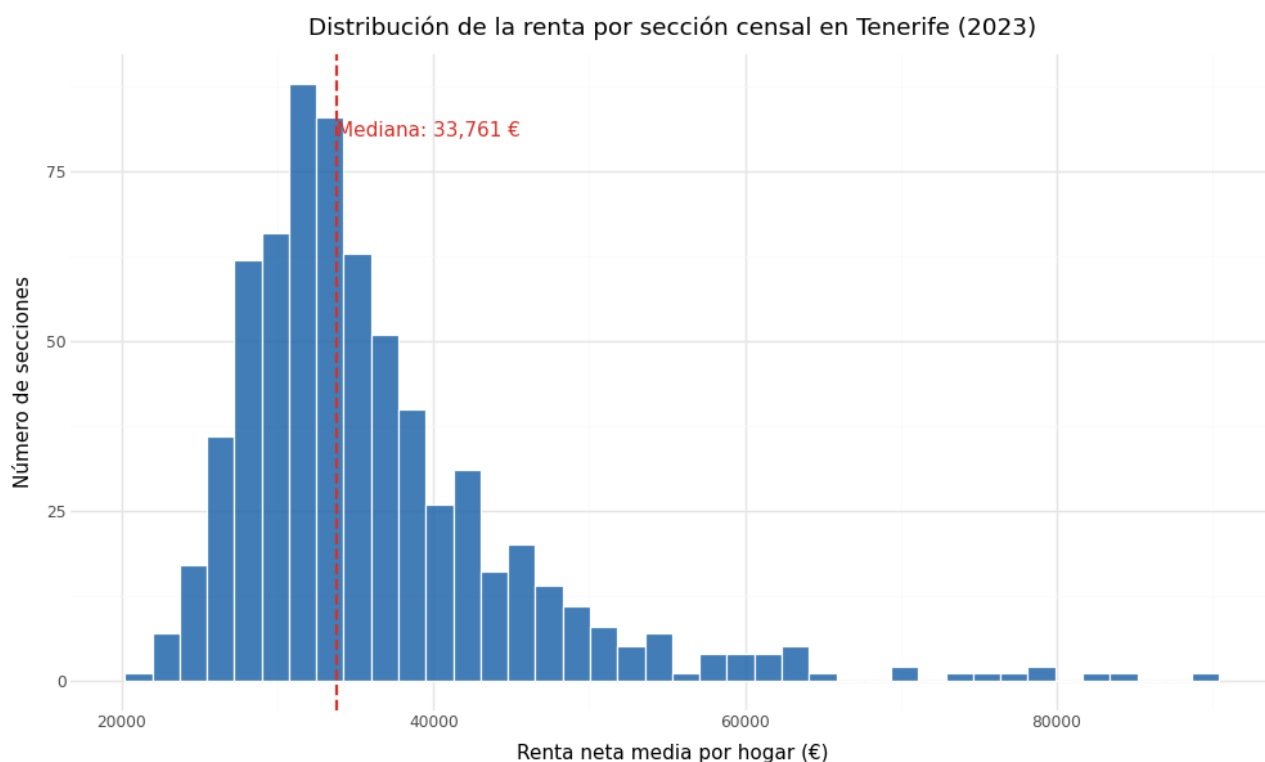


Figura 2.2: Histograma de distribución de la renta neta por sección censal.

- Datos: tabla con un registro por sección censal; columna `renta_neta`.
- Mapeos: `x = renta_neta`.
- Geometrías: `geom_histogram (bins=40)` + `geom_vline` para indicar la mediana.
- Escalas: eje X continuo en euros; eje Y de frecuencia absoluta.
- Coordenadas: cartesianas estándar.
- Facetas: ninguna.
- Tema: `theme_minimal()`.

El histograma revela la asimetría positiva (cola derecha larga) característica de las distribuciones de renta: la mayoría de las secciones se concentran en la parte baja, con pocas secciones de renta muy alta.

2.3. Waterfall: fuentes de ingresos

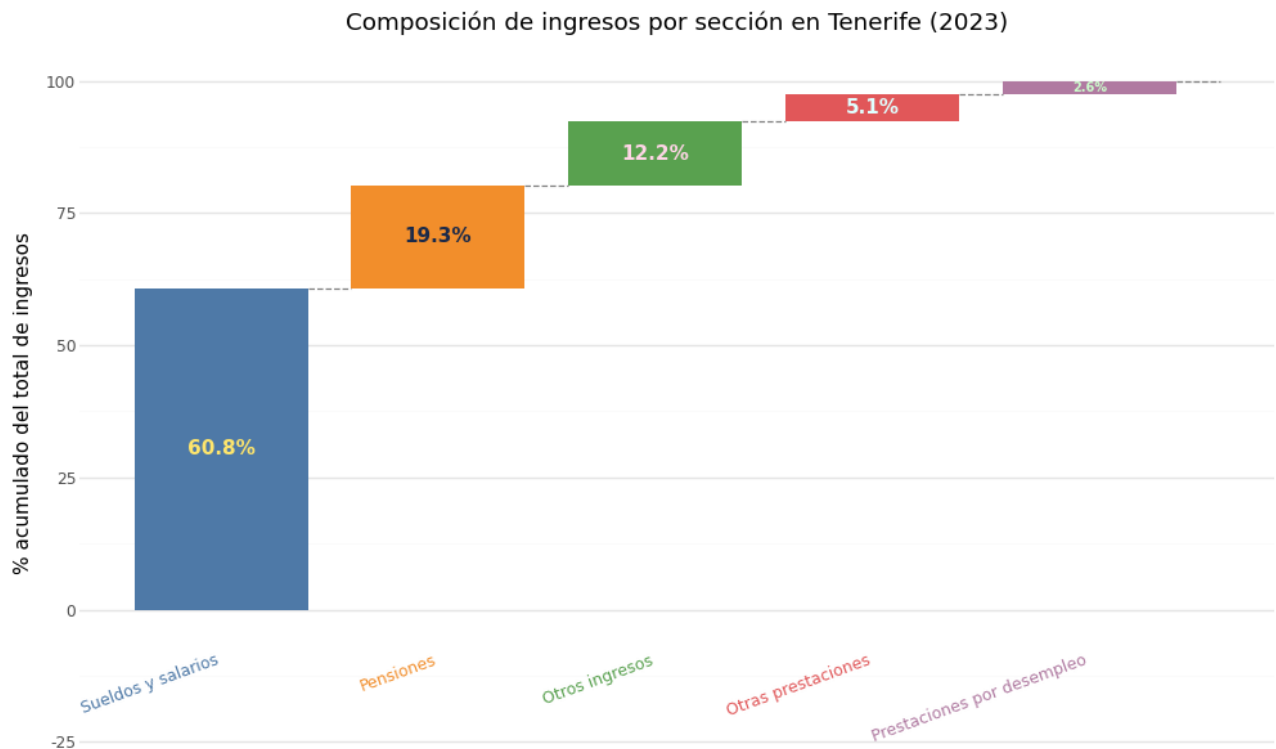


Figura 2.3: Waterfall de fuentes de ingresos: composición porcentual (2023).

- Datos: cinco filas, una por fuente de ingresos; columnas fuente y pct (porcentaje del total).
- Mapeos: `x` = fuente (categórica), `y` = pct, `fill` = fuente.
- Geometrías: `geom_col` con las barras apiladas en cascada (waterfall): cada barra comienza donde termina la anterior, de modo que la suma acumulada avanza de izquierda a derecha hasta el 100 %.
- Escalas: eje Y en porcentaje; paleta de colores divergente para distinguir fuentes activas (sueldos) de pasivas (pensiones, prestaciones).
- Coordenadas: cartesianas.
- Facetas: ninguna.
- Tema: etiquetas de porcentaje sobre cada barra con `geom_text`.

2.4. Waterfall: distribución por sector de actividad

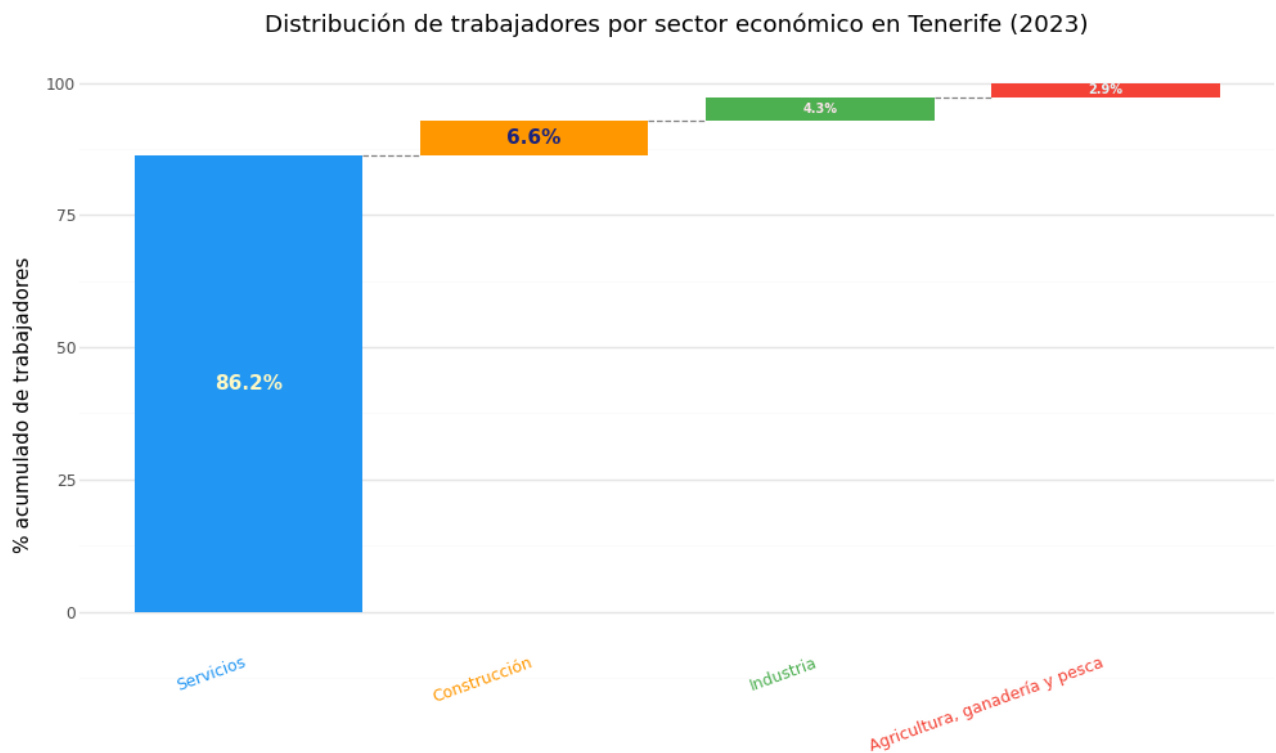


Figura 2.4: Waterfall de distribución de trabajadores por sector de actividad (2023).

Mismo esquema que el anterior pero para los cuatro sectores CNAE: Servicios, Industria, Construcción y Agricultura. El sector servicios domina ampliamente (más del 70 % de los asalariados), dato que da contexto a los gráficos de quintiles.

2.5. Diverging bar: brecha de género en ocupaciones

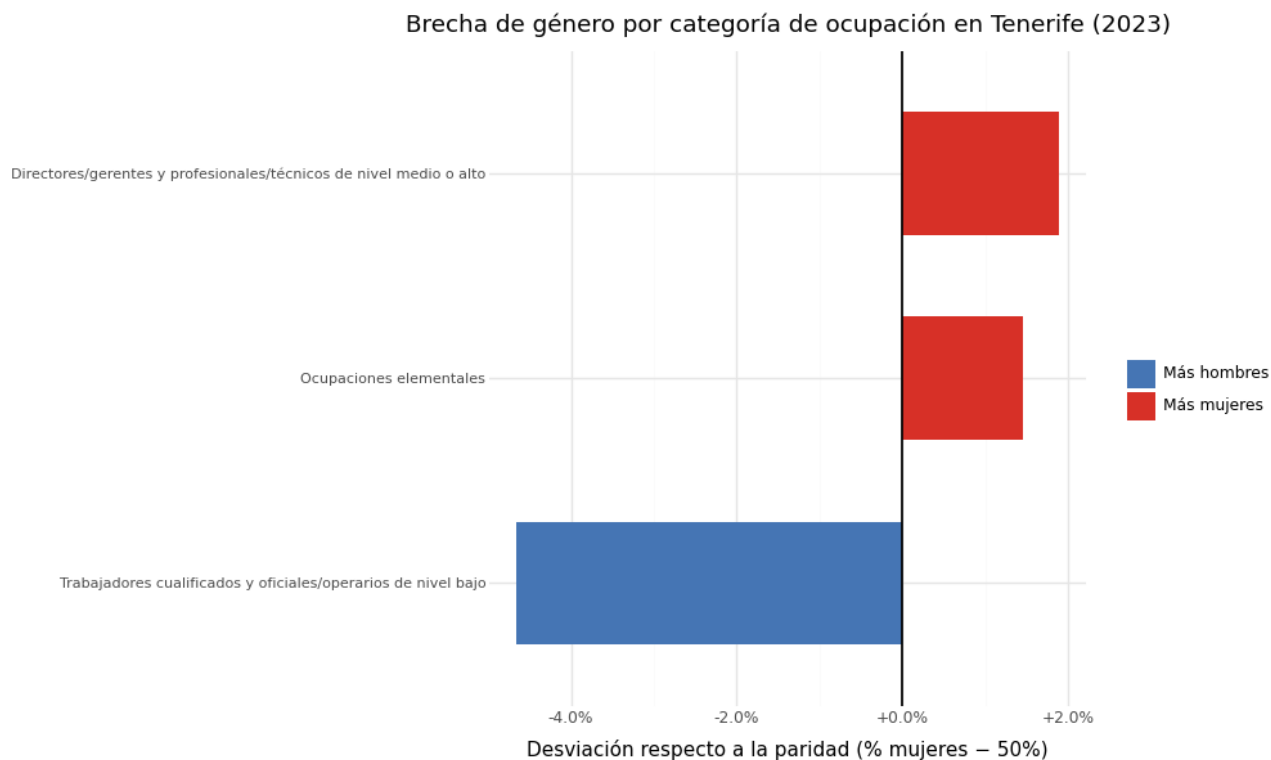


Figura 2.5: Desviación respecto a la paridad de género (50/50) por categoría de ocupación (2023).

- Datos: tabla con un registro por categoría de ocupación; columnas ocupacion, desviacion (porcentaje de mujeres menos 50) y mayoría ("Más mujeres/ "Más hombres").
- Mapeos:
 - x = ocupacion (categórica, ordenada por desviación)
 - y = desviacion (numérico; negativo = mayoría masculina, positivo = mayoría femenina)
 - fill = mayoría (dos colores: rojo para mayoría femenina, azul para masculina)
- Geometrías: `geom_col + geom_hline(yintercept=0)` para la línea de paridad.
- Escalas: eje Y con etiquetas en formato `+X.X % / -X.X %`; escala de color manual con dos valores.
- Coordenadas: `coord_flip()`.
- Facetas: ninguna.
- Tema: leyenda integrada en el gráfico; texto del eje Y reducido.

El diverging bar es el tipo de gráfico más adecuado para mostrar desviaciones respecto a un punto de referencia (aquí, la paridad perfecta al 50 %). Permite ver de un vistazo qué categorías están feminizadas o masculinizadas.

2.6. Slope chart: evolución de la renta 2021–2023

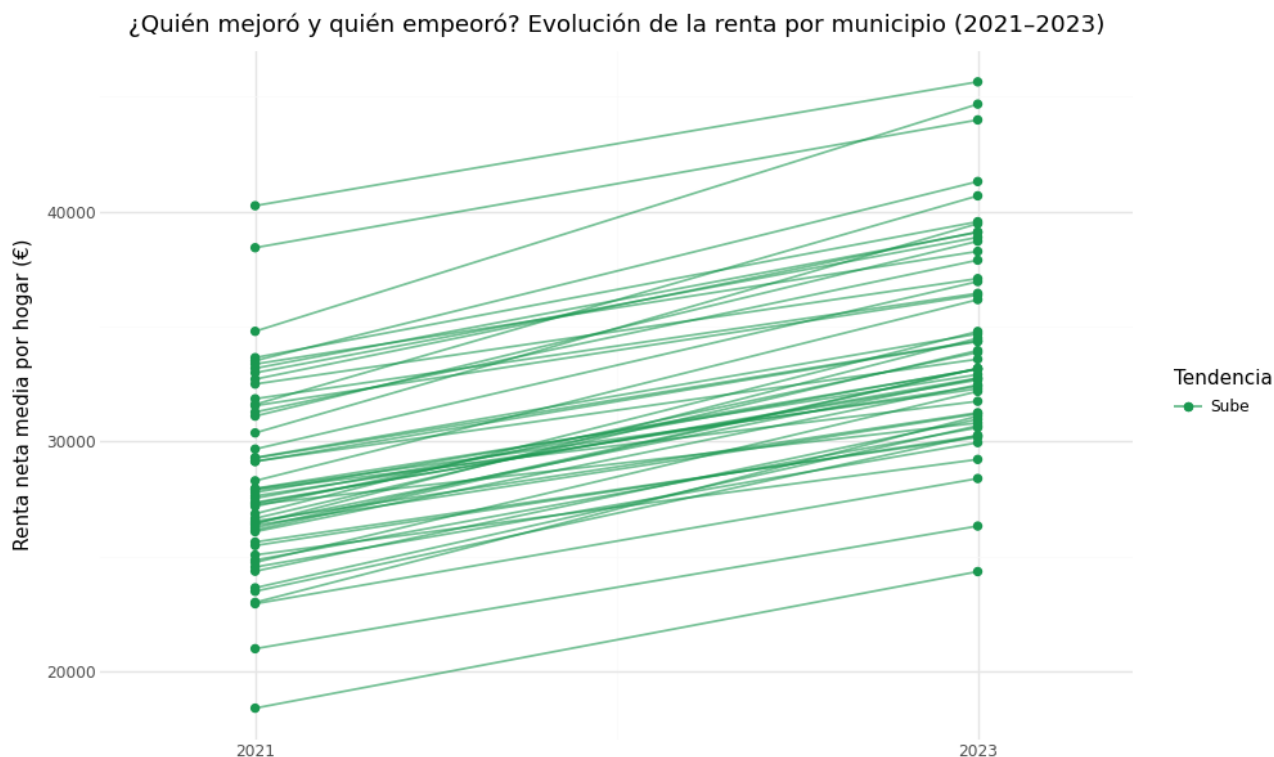


Figura 2.6: Slope chart de evolución de la renta media por municipio entre 2021 y 2023.

- Datos: tabla con un registro por municipio; columnas municipio, renta_2021 y renta_2023.
- Mapeos: x = año (dos valores: 2021 y 2023), y = renta_media, group = municipio, color = isla.
- Geometrías: geom_line (la pendiente entre los dos años) + geom_point en los extremos.
- Escalas: eje Y continuo en euros; eje X discreto con dos posiciones.
- Coordenadas: cartesianas.
- Facetas: ninguna; todos los municipios superpuestos.
- Tema: theme_minimal().

El slope chart es ideal para comparar el cambio relativo entre dos momentos: la inclinación de cada línea indica si un municipio ha ganado o perdido posición relativa. La densidad de líneas cruzadas señala cambios de orden en el ranking.

2.7. Box plots: desigualdad intramunicipal por isla

- Datos: un registro por sección censal con columnas municipio e isla. Se filtra por isla y se genera un PNG independiente por isla.
- Mapeos: x = municipio (categórica, ordenada por mediana), y = renta_neta.

- Geometrías: `geom_boxplot` con `outlier_size=0.8` (puntos pequeños para los valores atípicos).
- Escalas: eje Y continuo en euros.
- Coordenadas: `coord_flip()`.
- Facetas: ninguna (se generan cuatro archivos separados en lugar de un `facet_wrap()`).
- Tema: `theme_minimal()` sin leyenda; altura del gráfico escalada automáticamente según el número de municipios de la isla.

Generar un PNG por isla en lugar de un único gráfico con facetas permite adaptar la escala vertical a cada isla y evitar que los municipios de islas pequeñas queden aplastados.

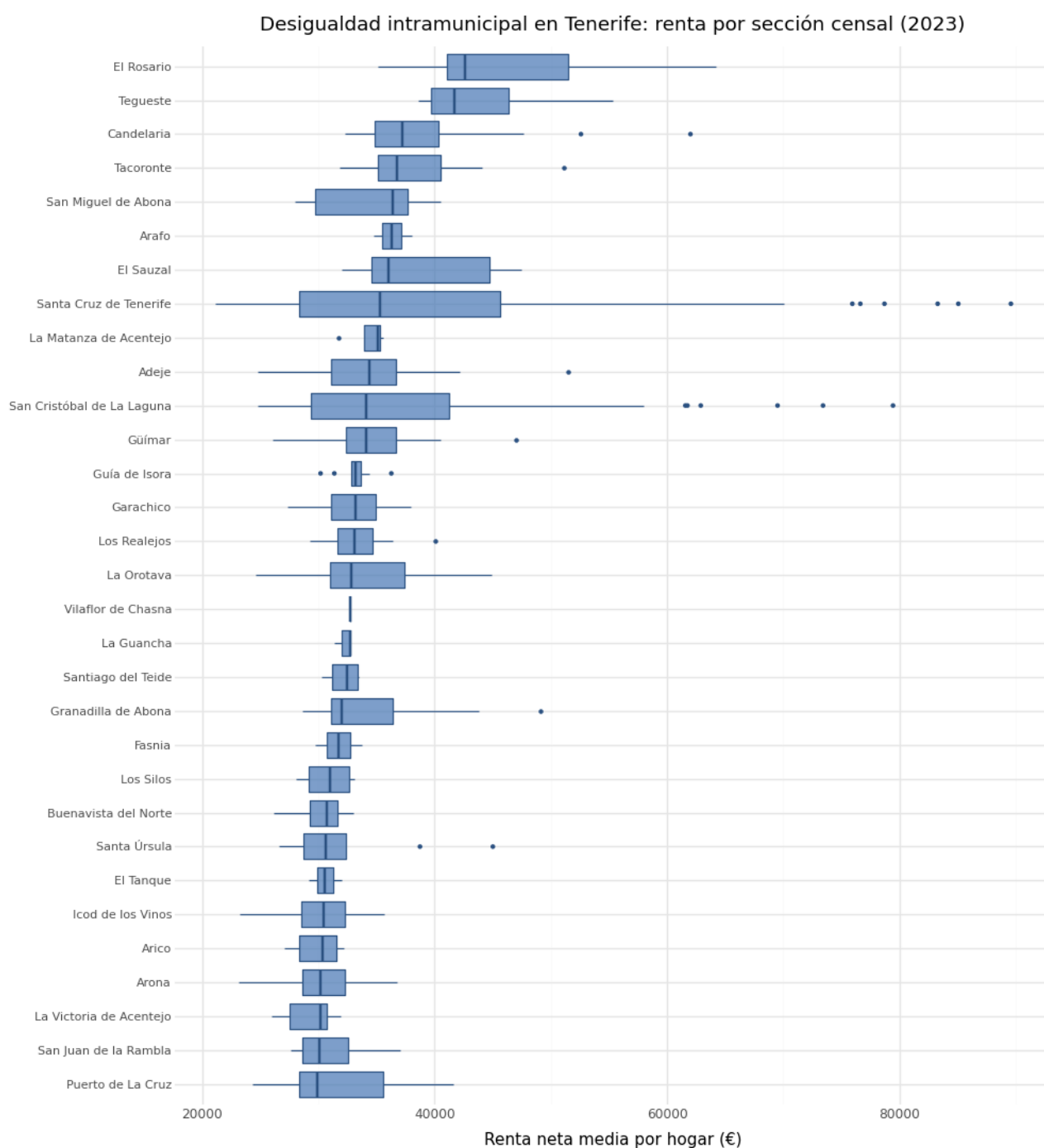
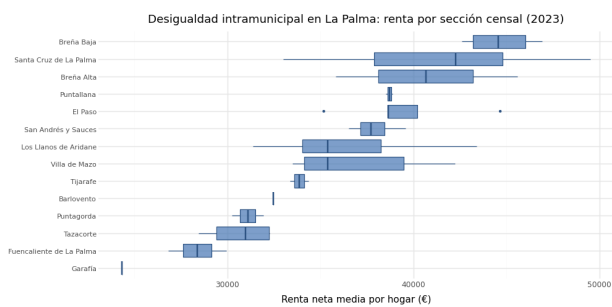
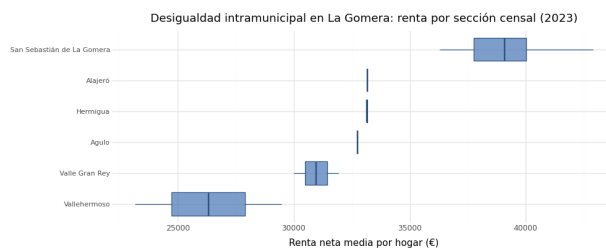


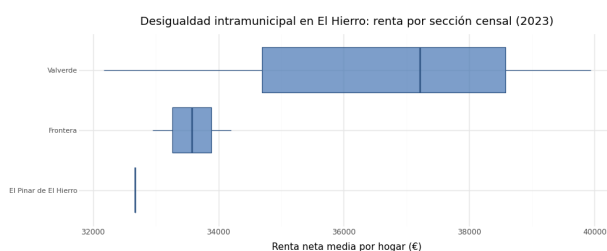
Figura 2.7: Desigualdad intramunicipal en Tenerife: renta por sección censal (2023).



(a) La Palma



(b) La Gomera



(c) El Hierro

Figura 2.8: Desigualdad intramunicipal en las islas menores: renta por sección censal (2023).

2.8. Scatter: estudios elementales vs. renta

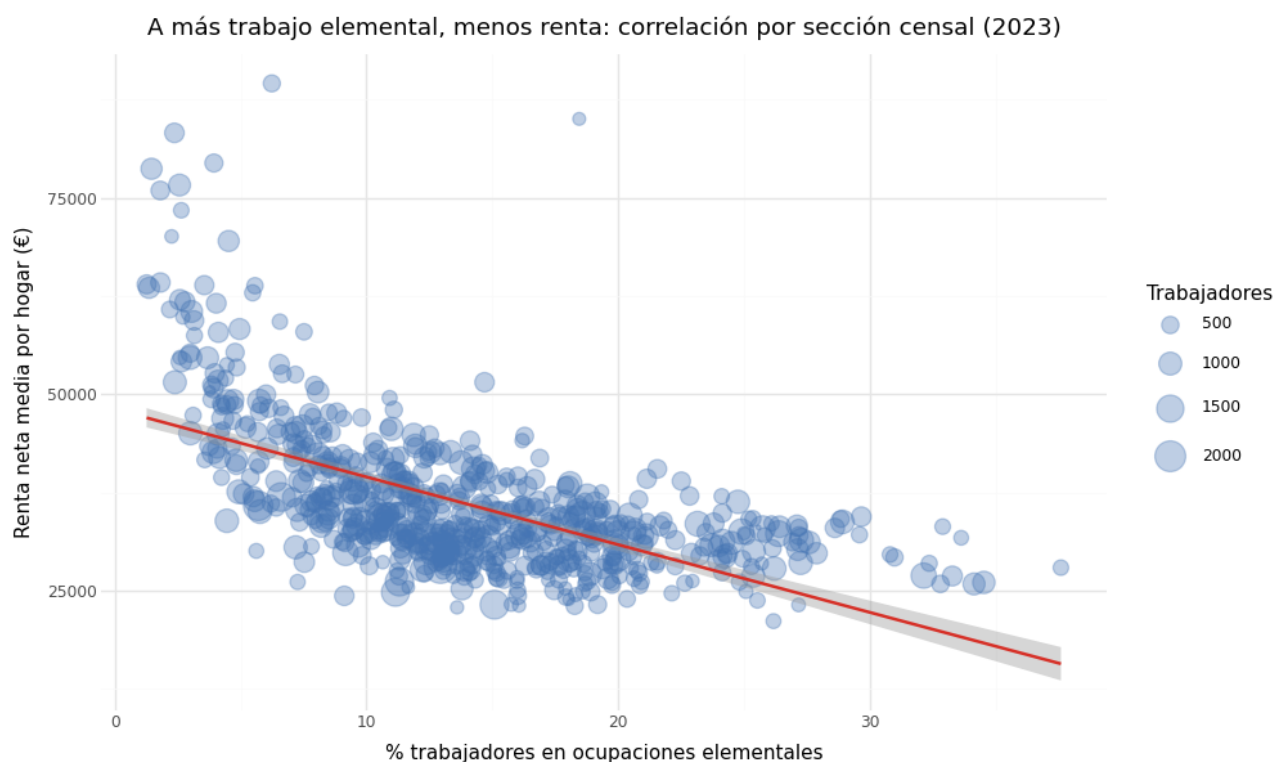


Figura 2.9: Relación entre porcentaje de trabajadores con nivel elemental y renta media municipal (2023).

- Datos: un registro por municipio; columnas pct_elementales (porcentaje de trabajadores con nivel formativo elemental) y renta_neta (renta media).

- Mapeos: `x = pct_elementales`, `y = renta_neta`, `label = municipio` (para identificar puntos extremos).
- Geometrías: `geom_point + geom_smooth(method='lm')` (regresión lineal) + `geom_text` para las etiquetas.
- Escalas: ambos ejes continuos; eje X en porcentaje.
- Coordenadas: cartesianas.
- Facetas: ninguna.
- Tema: `theme_minimal()`.

La correlación negativa esperada (a mayor porcentaje de trabajadores con estudios elementales, menor renta media) valida el supuesto narrativo del proyecto.

2.9. Grouped bar: distribución de actividad por quintil de renta

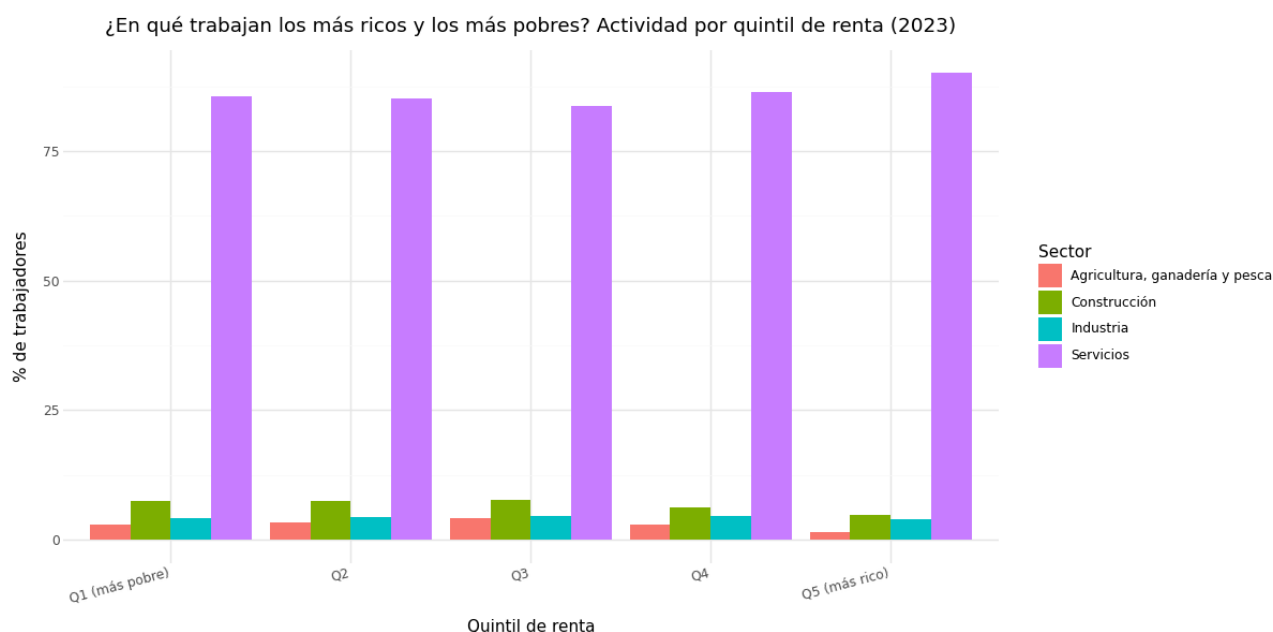


Figura 2.10: Distribución de trabajadores por sector de actividad y quintil de renta (2023).

- Datos: tabla con columnas quintil (1–5), actividad (sector CNAE) y pct (porcentaje de trabajadores).
- Mapeos: `x = quintil`, `y = pct`, `fill = actividad`.
- Geometrías: `geom_col(position='dodge')` (barras agrupadas, no apiladas).
- Escalas: eje Y en porcentaje; paleta de colores cualitativos para los cuatro sectores.
- Coordenadas: cartesianas; eje X discreto (quintil es una variable ordinal tratada como categórica).

- Facetas: ninguna.
- Tema: `theme_minimal()`.

2.10. Grouped bar: distribución de ocupación por quintil de renta

¿Qué ocupación tienen los más ricos y los más pobres? Por quintil de renta (2023)

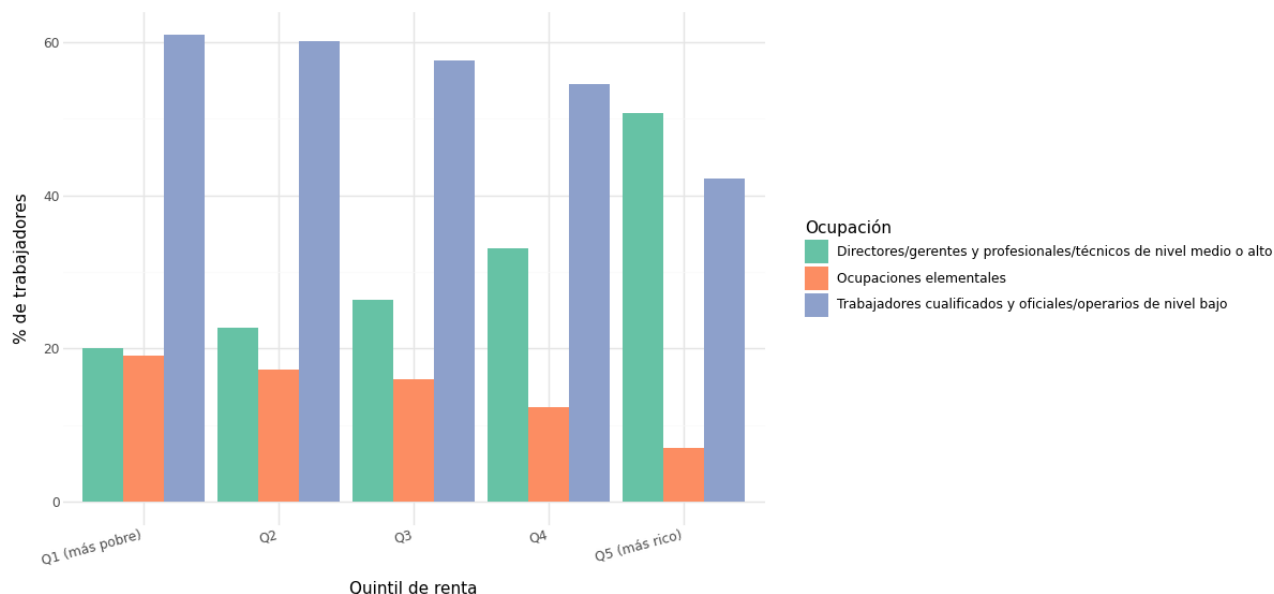


Figura 2.11: Distribución de trabajadores por categoría de ocupación y quintil de renta (2023).

Mismo esquema que el anterior, pero la variable de agrupación es la categoría de ocupación (CNO) en lugar del sector de actividad. Se incluyen las tres categorías más informativas desde el punto de vista de la desigualdad: directores/técnicos, trabajadores cualificados y ocupaciones elementales. Los colores siguen un esquema semafórico: verde para la ocupación de mayor renta, rojo para la de menor.

2.11. Heatmap: evolución de la renta municipal 2021–2023

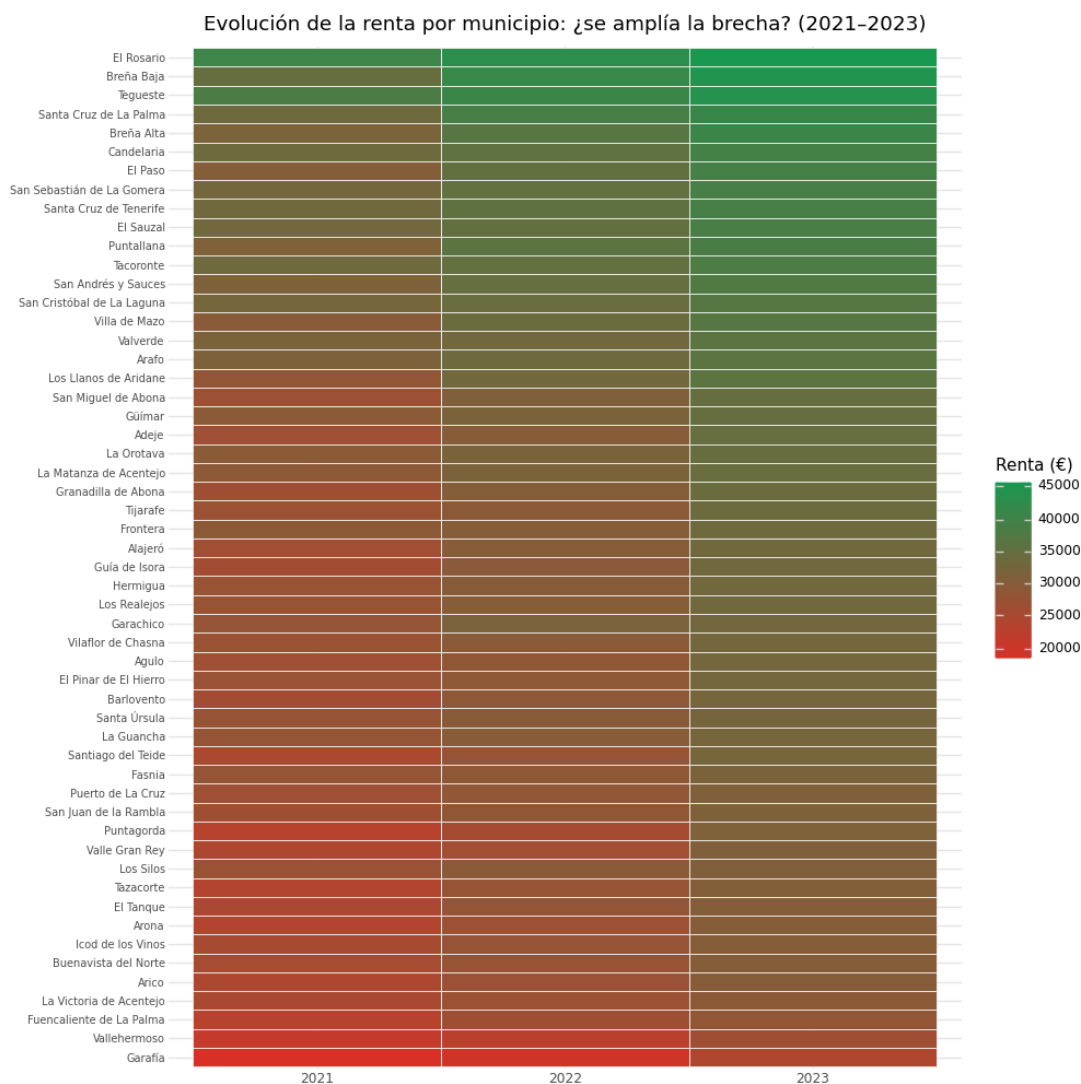


Figura 2.12: Heatmap de renta media normalizada por municipio y año (2021–2023).

- Datos: tabla con columnas municipio, año (2021, 2022, 2023) y renta_norm (renta normalizada en $[0,1]$ para poder comparar dentro del mismo municipio a lo largo del tiempo).
- Mapeos: $x = \text{año}$, $y = \text{municipio}$ (ordenado por renta media), $\text{fill} = \text{renta_norm}$.
- Geometrías: `geom_tile`.
- Escalas: escala de color divergente (RdYlGn): rojo para renta baja, verde para renta alta.
- Coordenadas: cartesianas.
- Facetas: ninguna.
- Tema: texto del eje Y pequeño; leyenda de color a la derecha.

2.12. Mapa interactivo: secciones censales

2.12.1. Técnica y datos

El mapa es un mapa coroplético (choropleth): cada polígono del territorio se colorea proporcionalmente al valor de una variable cuantitativa. En este caso los polígonos son las secciones censales de la provincia de Santa Cruz de Tenerife y la variable es la renta neta media por hogar (OBS_VALUE).

Los datos geométricos provienen de los ficheros GeoJSON del Instituto Geográfico Nacional (IGN), que delimitan las secciones censales a nivel de provincia. Estos polígonos se unen al dataframe de renta mediante el código de sección censal de siete dígitos, que identifica de forma única cada sección en España.

La librería Folium genera el mapa como un documento HTML autónomo que envuelve una instancia de Leaflet.js. Leaflet renderiza los polígonos en un canvas SVG sobre un tile base (OpenStreetMap por defecto) y gestiona las interacciones: zoom, paneo y hover sobre los polígonos.

2.12.2. Codificación visual

La variable de renta se mapea a color mediante una escala continua lineal (branca.LinearColormap) con la paleta YlOrRd (amarillo a rojo), donde el amarillo más claro corresponde a las secciones de menor renta y el rojo más oscuro a las de mayor renta. La escala se ajusta a los percentiles 5 y 95 de la distribución para reducir el efecto de valores atípicos en los extremos y maximizar el contraste en la zona central de la distribución.

Cada polígono incluye un tooltip que muestra el nombre del municipio, el código de sección censal y el valor de renta al pasar el ratón por encima, lo que permite identificar zonas de interés sin necesidad de una leyenda exhaustiva.

2.12.3. Diseño geoespacial vs. gramática de gráficos

A diferencia de los gráficos de Plotnine, este mapa no se puede describir completamente con la gramática de Wilkinson porque la variable geométrica (posición y forma de cada polígono) ya viene codificada en los datos GeoJSON y no es un mapeo libre. El único mapeo estético en sentido estricto es el del color. Sin embargo, los principios de diseño se aplican igualmente:

- Figura-Fondo: los polígonos coloreados destacan sobre el tile base neutro.
- Punto Focal: los clusters de color rojo intenso (zonas de alta renta) atraen la atención de inmediato y permiten identificar las áreas más ricas de la provincia.
- Proximidad: las secciones censales del mismo municipio son polígonos adyacentes, por lo que la homogeneidad de color dentro de un municipio se percibe sin etiquetas.

El mapa es la única visualización del proyecto en la que la dimensión espacial es el canal principal de codificación, lo que lo hace imprescindible para comunicar patrones de segregación residencial que no son perceptibles en los gráficos estadísticos.

3. Principios de diseño y gestalt

En la siguiente tabla se recoge para cada visualización qué principios de la gestalt o de diseño de visualización justifican las decisiones de codificación tomadas.

Gráfico	Principio(s)		Justificación
Lollipop ranking	Figura-Fondo, Focal	Punto	La línea delgada reduce el ruido visual (figura-fondo); el punto actúa como elemento focal que guía la lectura del valor preciso.
Histograma renta	Proximidad, Fondo	Figura-	Los bins adyacentes que comparten rango se perciben como grupo; el fondo blanco realza la distribución.
Waterfall fuentes	Continuidad, Todo	Parte-	Las barras encadenadas crean una línea de acumulación continua que comunica la composición del total; cada segmento es una parte del 100 %.
Waterfall actividades	Continuidad, Todo	Parte-	Mismo principio; la dominancia del sector servicios se percibe inmediatamente por la anchura relativa.
Diverging bar género	Destino Común, Simetría		Las barras que se extienden hacia lados opuestos convergen en el eje de paridad; la simetría rota evidencia el desequilibrio.
Slope chart	Cambio Temporal, Destino Común		Las líneas paralelas comunican continuidad temporal; las que se cruzan señalan cambios de orden.
Box plots (islas)	Distribución, Punto Focal		La caja muestra la distribución completa; la mediana como línea central actúa de punto focal para la comparación entre municipios.
Scatter	Correlación, Destino Común		Los puntos que forman una nube orientada negativamente señalan un destino común: a más educación elemental, menor renta.
Grouped bar quintiles	Magnitud, Proximidad		Las barras agrupadas permiten comparar magnitudes entre quintiles; la proximidad física de las barras del mismo quintil facilita la comparación intragrupo.
Grouped bar ocupación	Magnitud, Proximidad		Mismo principio aplicado a las categorías de ocupación.
Heatmap	Cambio Temporal, Similitud		El color codifica magnitud; celdas del mismo tono se perciben como similares. La organización temporal en columnas facilita ver la evolución.
Mapa interactivo	Figura-Fondo, Focal	Punto	El gradiente de color sobre fondo neutro enfoca la atención en las diferencias espaciales de renta.

Tabla 3.1: Principios de diseño y gestalt por visualización.

3.1. Recomendaciones de diseño aplicadas

A continuación se describen las recomendaciones de diseño más relevantes que se siguieron en el proyecto:

1. Relación datos/tinta (data-ink ratio). Todos los gráficos emplean `theme_minimal()` de Plotnine, que elimina el fondo gris, los bordes del panel y las rejillas secundarias. En los lollipops se usa una línea fina en lugar de barras rellenas para maximizar este ratio.
2. Ordenación por valor, no alfabética. En el ranking de municipios, el heatmap y los box plots, las categorías del eje Y se ordenan por la magnitud de la variable cuantitativa (renta mediana o media). Esto facilita la lectura comparativa y evita la búsqueda aleatoria.
3. Uso de `coord_flip()` para etiquetas largas. Los gráficos con etiquetas de municipio u ocupación se orientan horizontalmente mediante `coord_flip()` para evitar la rotación de texto y reducir la carga cognitiva del lector.
4. Paletas semánticas.
 - El diverging bar de género usa azul (mayoría masculina) y rojo (mayoría femenina), colores con convención social reconocible.
 - El grouped bar de ocupación usa verde (ocupaciones de alta renta) y rojo (ocupaciones elementales) siguiendo la convención semafórica.
 - El heatmap usa la paleta RdYlGn: rojo para valores bajos (situación desfavorable) y verde para altos.
5. Anotaciones y líneas de referencia. El diverging bar incluye una línea en $y=0$ (paridad exacta). El scatter incluye la recta de regresión. El histograma incluye una línea vertical en la mediana. Estas anotaciones anclan la interpretación sin necesidad de texto adicional.
6. Carga cognitiva limitada en variables categóricas. Ningún gráfico supera 10 categorías de color simultáneas (principio de límite de la memoria de trabajo). El check de activos `test_ocupacion_categorias` verifica programáticamente este límite.
7. Generación de archivos separados por subgrupo. Los box plots se generan como cuatro archivos PNG independientes (uno por isla) en lugar de un único panel con `facet_wrap`. Esto permite ajustar la escala vertical a cada isla y mantener la legibilidad de las etiquetas de municipio.

4. Arquitectura DataOps con Dagster

4.1. Motivación

El proyecto adopta una arquitectura DataOps basada en Dagster para automatizar la generación de los gráficos de forma reproducible, trazable y con garantías de calidad. Dagster organiza el trabajo como un grafo dirigido acíclico (DAG) de assets: cada función decorada con `@asset` declara explícitamente sus dependencias de datos, lo que permite al orquestador determinar el orden de ejecución y recalculan sólo los assets afectados por un cambio.

4.2. Grafo de assets

El pipeline se estructura en cuatro capas:

1. Carga (`*_load`): lectura de los datos crudos desde la API del INE o desde ficheros CSV en caché.
2. Limpieza (`*_clean`): normalización de tipos, filtrado de nulos y codificación de variables.
3. Datos analíticos (`data_*`): transformaciones específicas para cada gráfico (agrupaciones, pivotados, cálculos de quintiles, etc.).
4. Visualizaciones (`ranking_municipios`, `hist_renta`, `fuentes_ingresos`, ...): generación del archivo PNG o HTML final.

Listing 4.1: Cadena de dependencias en Dagster (extracto).

```
rentamedia_load
-> rentamedia_clean
    -> data_renta_municipio
        -> ranking_municipios    # PNG
    -> data_hist_renta
        -> hist_renta            # PNG
    -> data_slope_brecha
        -> slope_brecha          # PNG
    -> data_boxplot_intramunicipal
        -> boxplot_intramunicipal # 4 x PNG
    -> data_heatmap_renta
        -> heatmap_renta         # PNG
```

4.3. Generación de código con IA y caché de prompts

Los assets de visualización no contienen el código de Plotnine directamente. En su lugar, cada asset llama a la función auxiliar `render_ia_viz`, que:

1. Calcula el hash MD5 del prompt de la visualización (almacenado en `sources/*.hash`).
2. Si el hash ha cambiado, invoca la API de Claude para generar el código Python de la función `generar_plot(df)` y lo guarda en `scripts/*.py`.

3. Carga el módulo generado dinámicamente, llama a `generar_plot(df)` con los datos del asset, y guarda el resultado como PNG.

Este mecanismo garantiza que el código generado por la IA sea determinista: si el prompt no ha cambiado, se reutiliza el código en caché sin hacer ninguna llamada a la API. Sólo cuando el prompt se modifica (porque queremos un gráfico diferente) se regenera el script.

Listing 4.2: Función auxiliar de renderizado con caché de prompts.

```
def render_ia_viz(context, code: str, df, output_path: str,
                  width=10, height=6):
    namespace = {}
    exec(compile(code, "<ia_viz>", "exec"), namespace)
    plot = namespace["generar_plot"](df)
    plot.save(output_path, width=width, height=height, dpi=150)
    context.log.info(f"Guardado: {output_path}")
```

Listing 4.3: Patrón de asset con caché de prompt hash.

```
@asset
def ranking_municipios(context, prompt_ranking, data_renta_municipio):
    script_path = os.path.join(SOURCES_DIR, "ranking_municipios.py")
    hash_path = os.path.join(SOURCES_DIR, "ranking_municipios.hash")
    prompt_hash = hashlib.md5(prompt_ranking.encode()).hexdigest()

    if not os.path.exists(hash_path) or \
        open(hash_path).read().strip() != prompt_hash:
        code = llamar_api_ia(prompt_ranking)
        with open(script_path, "w") as f:
            f.write(code)
        with open(hash_path, "w") as f:
            f.write(prompt_hash)

    with open(script_path) as f:
        code = f.read()

    output_path = "plots/renta/ranking_municipios.png"
    render_ia_viz(context, code, data_renta_municipio,
                  output_path, width=12, height=14)
    return output_path
```

4.4. Beneficios de la arquitectura

- Reproducibilidad: dado el mismo conjunto de datos y el mismo prompt, el pipeline siempre produce el mismo PNG.
- Trazabilidad: Dagster registra metadatos de cada ejecución: tamaño del archivo generado, número de registros del dataframe de entrada, ruta del archivo, etc.
- Modularidad: cada step es independiente. Cambiar la transformación de datos de `data_renta_municipio` sólo implica re-ejecutar ese asset y los que dependen de él; los demás no se recalculan.

- Observabilidad: la interfaz web de Dagster muestra el grafo de dependencias, el historial de ejecuciones y los metadatos de cada asset en tiempo real.

5. Checks de calidad de assets

Cada asset importante cuenta con uno o más asset checks implementados en `lab_renta_checks.py`. Los checks son funciones decoradas con `@asset_check` que devuelven un `AssetCheckResult` con un campo `passed` y metadatos adicionales.

5.1. Clasificación de los checks

5.1.1. Checks de carga (integridad básica)

Check	Qué verifica
<code>test_rentamedia_no_nulos</code>	Porcentaje de nulos en <code>OBS_VALUE</code> <5 %
<code>test_actividad_sectores_completos</code>	Los 5 sectores de actividad económica están presentes
<code>test_ingresos_fuentes_completas</code>	Las 5 fuentes de ingresos están presentes

Tabla 5.1: Checks de integridad en los assets de carga.

5.1.2. Checks de limpieza

Check	Qué verifica
<code>test_rentamedia_rango_valores</code>	Renta neta en [3.000 euros, 200.000 euros]: rango razonable para España
<code>test_rentamedia_cobertura_temporal</code>	Los años 2021, 2022 y 2023 están presentes
<code>test_ingresos_conversion_decimal</code>	<code>OBS_VALUE</code> en [0, 100] tras conversión de porcentajes
<code>test_ocupacion_categorias</code>	<= 10 categorías de ocupación (límite cognitivo de visualización)

Tabla 5.2: Checks sobre los assets de limpieza.

5.1.3. Checks analíticos

Estos checks validan los supuestos narrativos del proyecto: si fallan, la historia que cuentan los gráficos pierde sustento estadístico.

Check	Qué verifica
test_cobertura_municipios	Al menos 40 de los 54 municipios tienen datos de renta
test_punto_focal_renta	Ratio renta máxima / mínima ≥ 1.5 (la desigualdad es suficiente para ser narrativa)
test_hist_dispersion	Coefficiente de variación de la renta > 0.15 (el histograma es informativo)
test_waterfall_suma_100	Las 5 fuentes de ingresos suman aprox. 100 %
test_piramide_categorias	El dataset tiene columna desviacion y ≥ 2 categorías
test_waterfall_actividades_suma_100	Los 4 sectores CNAE suman aprox. 100 %
test_slope_dos_anios	El slope chart tiene columnas renta_2021 y renta_2023 y ≥ 40 municipios
test_boxplot_cobertura	El boxplot tiene ≥ 40 municipios y ≥ 200 secciones censales
test_scatter_correlacion_negativa	Correlación entre % elementales y renta < -0.1
test_quintiles_completos	5 quintiles y ≥ 3 sectores de actividad
test_ocupacion_quintiles_completos	5 quintiles y ≥ 2 categorías de ocupación
test_heatmap_tres_anios	El heatmap cubre los 3 años y ≥ 40 municipios

Tabla 5.3: Checks analíticos que validan los supuestos narrativos del proyecto.

5.1.4. Checks de integridad de archivos generados

Para cada PNG generado existe un check que verifica que el archivo existe y tiene un tamaño mayor a 5 KB (lo que descarta archivos vacíos o corrompidos):

Listing 5.1: Función auxiliar compartida por todos los checks de archivo.

```
def _check_file(path, description, gestalt, min_size=5000):
    exists = os.path.exists(path)
    size = os.path.getsize(path) if exists else 0
    return AssetCheckResult(
        passed=bool(exists and size > min_size),
        metadata={
            "path": MetadataValue.path(path),
            "size_kb": MetadataValue.float(size / 1024),
            "description": description,
            "gestalt": gestalt,
        }
    )
```

5.2. Metadatos de los checks

Todos los checks incluyen metadatos estructurados que Dagster almacena en su base de datos de eventos:

- description: descripción legible del criterio de aceptación.
- gestalt: principio de diseño relacionado (por ejemplo, "Figura-Fondo", "Continuidad", "Parte-Todo").

- Métricas numéricas específicas (`n_municipios`, `correlacion`, `ratio_max_min`, etc.) que permiten monitorizar la evolución del dataset a lo largo del tiempo.

6. Dashboard interactivo

6.1. Motivación

Los gráficos estáticos en PNG son adecuados para un informe impreso o un documento PDF, pero no permiten la exploración libre de los datos ni la combinación narrativa de todas las visualizaciones en un único flujo de lectura. El dashboard de Streamlit (`dashboard.py`) cubre esa necesidad: agrupa los catorce gráficos del proyecto junto con el mapa interactivo, añade texto explicativo contextual y organiza el contenido en una secuencia que sigue el arco narrativo del análisis.

6.2. Estructura y narrativa

El dashboard se divide en seis secciones accesibles desde un menú de radio en la barra lateral (`st.sidebar`). El orden de las secciones refleja el orden lógico del análisis: primero el contexto espacial (mapa), luego la distribución general de la renta, después la composición económica, a continuación los patrones de desigualdad y género, luego el detalle intramunicipal y finalmente la relación con el nivel educativo y los quintiles.

Sección	Visualizaciones	Pregunta que responde
Mapa interactivo	Mapa coroplético Folium	¿Dónde están las zonas ricas y pobres?
Distribución de la renta	Lollipop ranking, histograma	¿Cuánta diferencia hay entre municipios?
Composición económica	Waterfall fuentes, waterfall actividades	¿De qué viven los habitantes?
Desigualdad y género	Slope chart, diverging bar	¿Ha cambiado la brecha? ¿Se reparte igual el trabajo precario?
Desigualdad intramunicipal	4 box plots (uno por isla)	¿Cuánta heterogeneidad hay dentro de cada municipio?
Educación y quintiles	Scatter, grouped bars	¿Qué perfil educativo y laboral tienen los más ricos?

Tabla 6.1: Secciones del dashboard y preguntas que responden.

Cada sección incluye un bloque de texto introductorio que contextualiza la visualización y un recuadro de conclusión (insight box) con el hallazgo principal. Ambos elementos se renderizan mediante `st.markdown` con HTML personalizado para controlar el estilo visual sin depender del tema por defecto de Streamlit.

6.3. Integración del mapa interactivo

La principal dificultad técnica del dashboard es incrustar el mapa Folium, que es un fichero HTML autónomo con JavaScript, dentro de una aplicación Streamlit. La solución es leer el

HTML generado y pasarlo a `st.components.v1.html`, que crea un `iframe` en el que el navegador ejecuta el código `Leaflet.js` del mapa:

Listing 6.1: Incorporación del mapa Folium en Streamlit.

```
def show_map():
    with open("plots/secciones_map.html", "r") as f:
        html = f.read()
    st.components.v1.html(html, height=600, scrolling=False)
```

El mapa funciona con todas sus capacidades interactivas (zoom, paneo, tooltip al hover) dentro del `iframe`. La altura se fija en 600 píxeles para que quepa en una pantalla de resolución estándar sin necesidad de hacer scroll dentro del componente.

6.4. Navegación y estilo

La navegación lateral usa `st.radio` para listar las seis secciones. Al seleccionar una sección, Streamlit re-ejecuta el script y la función correspondiente renderiza el contenido. Los gráficos estáticos se muestran con `st.image`, que los escala al ancho de la columna de contenido.

El estilo visual se personaliza con CSS inyectado mediante `st.markdown` con `unsafe_allow_html=True`. Se definen dos clases: `caption-box` (borde azul a la izquierda, para las descripciones de contexto) e `insight-box` (borde amarillo a la izquierda, para los hallazgos principales). Esto mantiene la coherencia visual entre secciones sin necesidad de imágenes decorativas.

6.5. Arranque

El dashboard se lanza en el puerto 8501 para no interferir con otras aplicaciones que puedan usar el puerto 8080 o el 80:

Listing 6.2: Arranque del dashboard.

```
streamlit run dashboard.py --server.port 8501
```

Una vez en marcha, la aplicación queda disponible en `http://localhost:8501`. Streamlit recarga automáticamente el dashboard cada vez que se guarda `dashboard.py`, lo que facilita iterar sobre el diseño sin reiniciar el servidor manualmente.

Apéndice: Prompts de generación de visualizaciones

Este apéndice recoge los prompts utilizados para que el modelo de lenguaje genere el código de cada visualización. Los prompts están diseñados para referenciar explícitamente los conceptos de la gramática de gráficos y las recomendaciones de diseño de la asignatura, de modo que el código generado siga los mismos principios que se estudian.

Consideraciones de diseño de los prompts

Cada prompt incluye los siguientes elementos:

- Tipo de gráfico y justificación: se indica el tipo de geometría (`geom_*`) y por qué es adecuado para los datos (por ejemplo, "un lollipop reduce la tinta sin perder precisión respecto a un gráfico de barras").
- Mapeos estéticos explícitos: se especifica qué variable va en qué `aes()`, siguiendo la terminología de la gramática de gráficos.
- Transformaciones de coordenadas: cuando se requiere `coord_flip()`, se indica explícitamente junto con el patrón correcto (`x=categorica, y=numérica`).
- Principios de gestalt: se mencionan los principios que debe activar el gráfico (por ejemplo, "punto focal en la cabeza del lollipop", "continuidad en la línea acumulada del waterfall").
- Recomendaciones de diseño: se incluyen instrucciones sobre paletas de colores semánticas, límite de categorías, ordenación por valor, etc.

Evaluación de los prompts

Los prompts cumplen las siguientes condiciones de calidad:

1. Referencian la gramática de gráficos: todos los prompts nombran explícitamente `aes()`, `geom_*`, `scale_*`, `coord_*` y `theme_*`.
2. Referencian principios de diseño: al menos un principio de gestalt o de diseño de visualización se menciona en cada prompt.
3. Especifican el schema de datos: cada prompt describe las columnas del dataframe de entrada (nombres y tipos) para que el modelo no haga suposiciones.
4. Incluyen restricciones de calidad: se indica el formato de salida (función `generar_plot(df)` que devuelve un objeto `ggplot`), el estilo de tema y las restricciones de accesibilidad.

Prompts seleccionados

Prompt: ranking de municipios (lollipop)

Listing 6.3: Prompt para el lollipop de ranking.

Genera una función `generar_plot(df)` que cree un lollipop chart con `plotnine`. El dataframe `df` tiene columnas: `municipio` (str), `renta_media` (float), `isla` (str).

Mapeos `aes()`: `x=municipio` (ordenado por `renta_media` de menor a mayor), `y=renta_media`, `color=isla`. Usa `geom_segment` desde `ymin=0` hasta `yend` y `geom_point` para la cabeza. Aplica `coord_flip()` para que los nombres de `municipio` se lean horizontalmente. Usa `theme_minimal()` y etiquetas del eje Y en formato de miles de euros.

Principios de gestalt: Figura-Fondo (línea delgada reduce tinta), Punto Focal (`geom_point` guía la lectura del valor).

Prompt: diverging bar de género en ocupaciones

Listing 6.4: Prompt para el diverging bar de brecha de género.

Genera una función `generar_plot(df)` con `plotnine`. El dataframe tiene columnas: `ocupacion` (str), `desviacion` (float, negativo=mayoría masculina, positivo=mayoría femenina), `mayoria` (str).

Mapeos: `x=ocupacion` (pd.Categorical ordenado por `desviacion`), `y=desviacion`, `fill=mayoria`. Usa `geom_col(width=0.6)` + `coord_flip()` + `geom_hline(yintercept=0)`. IMPORTANTE: `x` debe ser la variable categórica y `y` la numérica, no al revés. Usa `scale_fill_manual` con azul para "Más hombres" y rojo para "Más mujeres" (convención de género). Formato del eje Y: etiquetas con signo (+/-) y símbolo %.

Principios: Destino Común (las barras de categorías masculinizadas apuntan en la misma dirección), Simetría (la asimetría real contrasta con la simetría esperada en la paridad).

Prompt: box plots intramunicipal

Listing 6.5: Prompt para el box plot de desigualdad intramunicipal.

Genera una función `generar_plot(df)` con `plotnine`. El dataframe tiene columnas: `municipio` (str), `renta_neta` (float), `isla` (str). El dataframe ya viene filtrado para una sola isla.

Ordena los municipios por `renta_neta` mediana (usa `pd.Categorical` con las categorías ordenadas). Mapeos: `x=municipio`, `y=renta_neta`. Geometría: `geom_boxplot` con `outlier_size=0.8` para no saturar. Aplica `coord_flip()`. El título debe incluir el nombre de la isla (usa `df['isla'].iloc[0]`). Usa `theme_minimal()` con `legend_position='none'`.

Principios: Distribución (la caja muestra varianza intramunicipal), Punto Focal (la mediana como línea central del boxplot).

Prompt: grouped bar por quintiles de actividad

Listing 6.6: Prompt para el grouped bar de actividad por quintil.

```
Genera una función generar_plot(df) con plotnine. El dataframe tiene
columnas: quintil (int, 1-5), actividad (str, sector CNAE), pct (float).
```

```
Convierte quintil a str antes de usarlo. Mapeos: x=quintil, y=pct,
fill=actividad. Geometría: geom_col(position='dodge'). Usa
theme_minimal(). Las barras deben estar agrupadas (dodge), no apiladas.
Eje Y en porcentaje.
```

```
Principios: Magnitud (la altura de las barras codifica proporción),
Proximidad (las barras del mismo quintil se perciben como grupo).
```

Los prompts completos para todos los gráficos están almacenados en los assets `prompt_*` del archivo `lab_renta.py` y en los ficheros de caché `sources/*`.hash.

Bibliografía

- [1] Wilkinson, L. (2005). The Grammar of Graphics (2nd ed.). Springer.
- [2] Kibirige, H. et al. (2022). plotnine: A grammar of graphics for Python. Journal of Open Source Software, 7(74), 3523.
- [3] Few, S. (2012). Show Me the Numbers: Designing Tables and Graphs to Enlighten (2nd ed.). Analytics Press.
- [4] Cairo, A. (2016). The Truthful Art: Data, Charts, and Maps for Communication. New Riders.
- [5] Elementl, Inc. (2023). Dagster: A data orchestrator for machine learning, analytics, and ETL. <https://dagster.io>
- [6] Instituto Nacional de Estadística (2024). Atlas de distribución de renta de los hogares. https://www.ine.es/experimental/atlas/experimental_atlas.htm